Algorithms for genomic data analysis

Assignment 1: read mapping

winter semester 2025/2026

Magdalena Czapiewska

January 10, 2026

Step 1: Find **minimizers** and their **positions** in the **reference sequence**

Minimizer = k-mer with the smallest hash value within a window of w consecutive k-mers

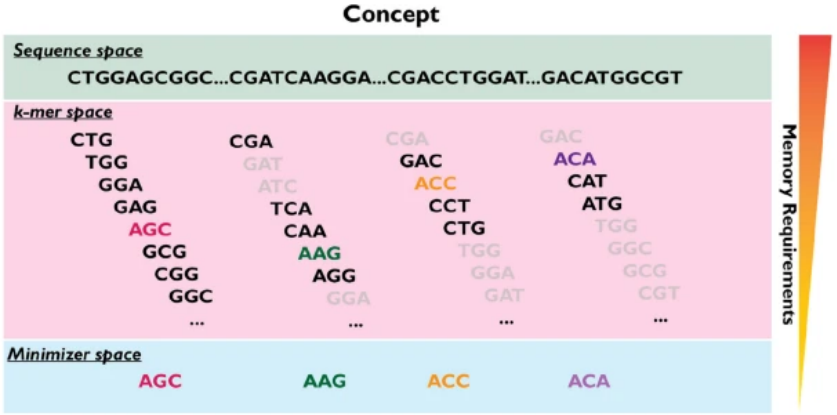

Figure: Source: Ndiaye et al. 2024 "When less is more: sketching with minimizers in genomics"

Input and result

- **Input:** a single read sequence
- **Result:** a *map* of minimizers (smallest k-mer hashes in sliding windows) to their positions in the sequence

Step 1: Find minimizers and their positions in the reference sequence - details

Sequence value encoding

- Each k-mer is encoded as a **2-bit integer** per **base**: A=00, C=01, G=10, T=11
- Example: `encode_kmer("ACG")` → 0b000110

Rolling computation of k-mer values

Compute value of the **first k-mer**. Then go along the sequence and for each new base:

- **update** k-mer value by **shifting previous value left** 2 bits and **adding the new base**
- keep only the last k-mer bits using a **mask**

Step 1: Find minimizers and their positions in the reference sequence - details

Choosing k and w

- **k = length of k-mer = 15**
- **w = window size for minimizer selection = 10**

These are the **default values** used in **Minimap2** tool. I also tried setting **k = 31 and w = 15**, because:

- value of 31-mer would fit into a **64-bit integer**
- I wanted to **reduce the number of minimizers matching** between the reference sequence and the read to have **less candidate positions**

but with read **error rate 5-10%** some reads were **unmappable** when using such large k (no matching minimizers).

Step 1: Find minimizers and their positions in the reference sequence - details

Algorithm overview

Maintain a **deque** storing **candidate k-mer hashes** and their **positions** within the current window of size w.
For each k-mer:

- **Remove** k-mers from the **front** of the deque that are **outside the current window**
- **Remove** k-mers from the **back** of the deque that are **larger** than the current hash (cannot be a minimizer)
- Append the current k-mer hash and its position to the deque
- If the window is full (i.e., at least w k-mers processed), the smallest hash in the deque (in the front) is the current minimizer
- Store **all positions** with the same minimal hash in a map: `minimizer hash → list of positions`
- **Avoid duplicate positions** for the same minimizer by tracking the last added values set that is cleared when minimizer value for a window differs from the previous one

Result: a map `minimizer hash → list of positions`

Step 2: Find minimizers and their positions in the read sequence. Match minimizers

For each read:

- Find **minimizers of the read** using the same k and w parameters as for the reference
- Find **matching minimizers between the read and the reference** sequence: go through minimizers in the read, and if the minimizer occurs in the reference once, **collect a pair of positions (read position, reference position)**. If the minimizer occurs in the reference **more than once**, **reject** this minimizer (I will later explain why).
- Calculate the **offset** for each pair:

$$\text{offset} = \text{position in reference} - \text{position in read}$$

and add it to a **list of offsets**.

Step 2: Find minimizers and their positions in the read sequence. Match minimizers

Offsets as relevant information

For a minimizer that occurs in a read and once in a reference, we compute:

$$\text{offset} = \text{position in reference} - \text{position in read}$$

- Offsets indicate the **relative position** of the read within the reference sequence. If the **read originates from the reference** and **starts at position x** in the reference, then a minimizer starting at position x+i in the reference will likely start at position i in the read (assuming it is detected despite sequencing errors).
- If multiple minimizers share the same or similar offset, they likely belong to the **same candidate mapping region**.
- This allows us to detect **clusters of similar offsets**, which represent probable locations where the read aligns. The larger the cluster, the higher the probability that the offset corresponds to the true read position in the reference.

Step 3: Find the most abundant offset cluster

Offset clustering to identify candidate mapping regions

- **Offsets from matching minimizers (minimizers that come from a true position of a read in reference)** are not identical due to sequencing errors. We allow differences up to **20% of read length** (error rate $\sim$ 20% of read length) when grouping offsets into clusters.
- Algorithm for clustering:
  1. **Sort all offsets** in ascending order.
  2. Slide a **window of size = 20% of read length** over the offsets.
  3. **Count the number of offsets** within each window.
  4. **Track the windows** with the **largest count of offsets** (most abundant clusters).
  5. Record the **smallest offset** in each of the best clusters as candidate starting positions for dynamic programming alignment.
- This step **reduces the number of candidate regions** for the costly **DP alignment**, focusing only on the most probable mapping positions.

Step 3: Find the most abundant offset cluster

Why do I ignore minimizers that occur more than once in the reference sequence?

Some **regions of a reference sequence** consist of short **motifs that repeat** many times (e.g., AAAT). A minimizer of length 15 taken from such a region may look like:

`AAATAAATAAATAAA`

Assume this minimizer appears once in the read. In the reference, however, it may appear at many positions, each yielding a different offset (e.g., offsets differing by 4).

In such a case, it may happen that **the most abundant offset cluster** within the allowed tolerance (offsets within distance 20% of read lenth) is actually produced by these **repeated minimizers** — not
by many distinct minimizers that all originate from the true mapping location of the read.

By removing minimizers that occur more than once in the reference, we avoid forming such **misleading clusters**. This does not harm the performance of the algorithm: we still detect **enough unique, correctly positioned minimizers** to identify reliable candidate mapping regions.

Step 4: Perform dynamic programming (DP) alignment in candidate regions

Align read to candidate reference regions

For **each cluster** offset (candidate region):

- Define a **reference region** around the offset:

  $$\text{win size} = 20\% \text{ read length}$$
  $$\text{candidate_start} = \max(0, \text{offset} - \text{win size})$$
  $$\text{candidate_end} = \min(\text{offset} + \text{read len} + \text{win size}, \text{reference len})$$

- **Extract the reference sequence** in this region.
- Perform **edit-distance dynamic programming** (DP) alignment of the read against the reference region.
- Compute **global alignment positions** by adding the candidate start offset to the **local alignment start and end** obtained from the DP matrix.
- Note: "global" here refers to positions in the **entire reference sequence**, while "local" refers to positions within the **extracted reference fragment**. This terminology does **not** refer to global or local alignment modes in dynamic programming.

Select the alignment with the **lowest edit distance** among all candidate clusters.
If edit distance $< 20\%$ of read length, accept the alignment; otherwise, mark the read as **unmappable**.

Step 4: Dynamic programming details

We align **the entire read** to **the candidate region** of the reference. The DP matrix is initialized like in **approximate string matching (pattern in text)**, **without** imposing a **penalty** for starting the alignment **away from the beginning** of the reference fragment.

													T									
		ϵ	A	A	C	C	C	T	A	T	G	T	C	A	T	G	C	C	T	T	G	G	A
	ϵ	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	T	1																					
	A	2																					
	C	3																					
	G	4																					
P	T	5																					
	C	6																					
	A	7																					
	G	8																					
	C	9																					

$$D[i,j] = \min \begin{cases} D[i-1,j]+1 & \text{upper} \\ D[i,j-1]+1 & \text{left} \\ D[i-1,j-1]+\delta(x[i-1],y[j-1]) & \text{upper-left} \end{cases}$$

Figure: Ben Langmead: Approximate Matching